

# In-Network IoT Attack Detection Using Principal Component Analysis and Machine Learning Models on P4 Programmable Data Planes

Viet Anh Phan<sup>1</sup>[0009-0003-1787-8063]✉, Mingyu Ma<sup>2,4</sup>[0009-0005-7045-4096],  
Tung V. Doan<sup>2,4</sup>, Giang T. Nguyen<sup>2,4</sup>[0000-0001-7008-1537],  
Frank H.P. Fitzek<sup>3,4</sup>[0000-0001-8469-9573], Jan Jerabek<sup>1</sup>[0000-0001-9487-5024],  
Lukas Malina<sup>1</sup>[0000-0002-7208-2514], and Jiri Hosek<sup>1</sup>[0000-0002-8382-9185]

<sup>1</sup> Department of Telecommunications, Brno University of Technology, Czech Republic  
{243760, jerabekj, malina, hosek}@vut.cz

<sup>2</sup> Haptic Communication Systems, TU Dresden, Germany

<sup>3</sup> Deutsche Telekom Chair of Communication Networks, TU Dresden, Germany

<sup>4</sup> Centre for Tactile Internet with Human-in-the-Loop (CeTI), TU Dresden, Germany  
{mingyu.ma, tung.doan\_van, giang.nguyen, frank.fitzek}@tu-dresden.de

**Abstract.** IoT gateways are critical vantage points for detecting attacks across heterogeneous, resource-constrained devices, but conventional intrusion detection that mirrors traffic to an external classifier adds latency and bandwidth overhead that line-rate IoT deployments cannot tolerate. P4 programmable switches address this gap: tree-based classifiers already run directly in the data plane at line rate, but Principal Component Analysis (PCA) needs multiply-accumulate operations unavailable in match-action pipelines, so PCA-projected features are typically computed off-switch before classification. This paper presents the first P4 intrusion-detection pipeline that performs PCA projection and tree classification under a single range-match abstraction, and evaluates the design on a Raspberry Pi 4 running BMv2 (`simple_switch_grpc`). Across 32 deployed configurations covering a Decision Tree and a 4-tree Random Forest with and without PCA, the raw range-match classifier achieves the highest in-distribution macro F1 on CIC-IoT 2023 (98.47%), yet degrades sharply on a disjoint Mu-IoT capture (15.57% macro F1) because port associations learned on the source dataset do not generalize. The recommended PCA + Decision Tree at  $k=7$  achieves 95.79% macro F1 in distribution and 48.31% on the disjoint capture, a 32.74 percentage-point cross-dataset gain at the cost of only 2.68 percentage points in distribution, making strong in-distribution accuracy and cross-dataset robustness jointly attainable. The recommended operating point clears the Tofino-1 per-stage match-key budget; stage-count compaction is the identified path to full Tofino-1 deployment.

**Keywords:** P4 programmable data plane · In-network machine learning · IoT · Intrusion detection system · Principal Component Analysis · Decision tree · Random forest · Range match tables · Bidirectional flow features · Line-rate classification

## 1 Introduction

The Internet of Things (IoT) connects an increasing number of resource-constrained devices through gateway switches that aggregate heterogeneous traffic before it enters the wider network. These gateways are a natural defensive vantage point because every flow from the protected segment passes through them, and they are also a critical bottleneck: an intrusion-detection system (IDS) that mirrors traffic off the forwarding path, or delegates the classification decision to an external controller, introduces latency and bandwidth overhead that traditional middlebox and SDN deployments cannot eliminate [7,19]. P4 programmable data planes lift this restriction by letting the switch parse, track, and classify flows in the pipeline at line rate, without recirculation or controller round trips. In-network ML detection has therefore become an active research area [20,11,23,1].

The same pipeline must run on two very different targets. At the IoT-gateway tier, P4 has reached small-footprint hardware through platforms such as P4Pi [10], which brings programmable P4 pipelines to the Raspberry Pi board; P4Pir [21] has further shown that DT and RF classifiers are feasible at this tier. All experiments in this paper are conducted on a Raspberry Pi 4 running BMv2 (`simple_switch_grpc`) with the v1model architecture, demonstrating the pipeline on low-cost IoT-gateway hardware. At higher tiers, where industrial IoT and cyber-physical workloads require deterministic latency, programmable ASICs such as Intel Tofino are the target [1]. Both tiers expose the same constraint: limited on-chip memory and a fixed number of pipeline stages restrict how expressive the classifier can be, so a P4 IDS pipeline that scales gracefully must keep its match-action footprint small.

Despite this progress, the existing body of work reveals five persistent gaps. *First*, most in-network ML systems rely on exact-match encoding, where the classifier table stores one entry per unique discretized training feature vector [11,23,22]. Coverage is therefore restricted to feature combinations actually observed during training, and the number of entries scales with the granularity of the discretization grid and the size of the training set  $N$ , so adding training samples or refining feature granularity grows the table proportionally and stresses on-chip memory budgets [2]. *Second*, the majority of P4-based detection systems perform only binary classification, which limits their utility for security teams that must triage diverse threats. *Third*, extracting bidirectional flow statistics entirely in the data plane remains underexplored; most systems rely on unidirectional or packet-level features, or offload feature computation to an external server. *Fourth*, dimensionality reduction has been treated in isolation from the downstream classifier: although Planter [23] supports PCA as a standalone module, no prior work integrates PCA with a downstream classifier under a single range-match encoding, and no prior work makes the trade-off between a raw-feature classifier and a PCA-projected one explicit on hardware-relevant axes (per-entry match-key width, classifier-table size, and pipeline-stage placement). *Fifth*, translating a trained ML model into a deployable P4 program typically

requires significant manual effort across separate tools for training, feature selection, table generation, and code synthesis.

This paper addresses all five gaps through the following contributions:

- DT and RF classifiers are deployed as range-match P4 entries rather than exact-match entries, representing each tree leaf as a single rule with per-feature ranges. This naturally expresses the axis-aligned splits of a decision tree and yields the strongest in-distribution operating point of the pipeline (raw range-match DT, macro F1 98.47%, 5 119 entries on CIC-IoT 2023).
- PCA is integrated as an optional dimensionality-reduction stage that is itself encoded as range-match P4 tables and feeds the downstream classifier through the same encoding. To the best of the authors’ knowledge, this is the first integration of PCA with a downstream classifier under a single range-match encoding in a P4 data plane. The trade-off introduced by this stage is characterized in detail: PCA exposes a tunable match-key dial via  $k$  (from 32 bits at  $k=1$  up to 480 bits at  $k=15$ , versus the 256-bit raw composite key) and improves cross-dataset robustness, at the cost of roughly 2 times more classifier entries than the raw-feature pipeline.
- Multiclass classification is performed directly in the data plane on the CIC-IoT 2023 dataset [15] (Benign, DoS, Brute Force, Reconnaissance), in contrast to the predominantly binary detectors in prior work [9,14,11].
- The pipeline extracts 18 bidirectional flow features in the data plane across six categories (protocol, ports, timing, volume, TCP flags, TCP window), substantially richer than the unidirectional or packet-level features used by prior in-network approaches [16,1,21].
- An automated end-to-end pipeline is provided that covers feature extraction, model training, optional PCA projection, table-entry generation, and P4 code synthesis, deployed and evaluated on a Raspberry Pi 4, including a *cross-dataset* transfer (from CIC-IoT 2023 to Mu-IoT) to quantify how each encoding generalizes beyond its training distribution.

The full source code, training pipeline, and table-generation scripts used in this paper are publicly available at <https://github.com/vafekt/p4sec>.

## 2 Related Work

Prior work is analyzed along five dimensions: inference location, table encoding, feature extraction, classification granularity, and use of dimensionality reduction. Table 1 provides the structured comparison.

**In-network ML inference:** Xiong and Zilberman [20] first mapped DT, SVM, Naive Bayes, and K-means to match-action tables on BMv2 and NetFPGA SUME with a small set of packet-level features. IIsy [22] added a hybrid Tofino architecture forwarding low-confidence packets to a backend; its discretized-feature encoding allocates one entry per discretized value, so table sizes scale with the number of distinct feature values. Planter [23] unified eleven ML algorithms

under three mapping methodologies (direct, encode-based, and lookup) and includes PCA as a standalone module. As an ML-mapping framework it does not include a feature-extraction stage; it consumes pre-computed CSV features and does not integrate PCA with a downstream classifier under a single range-match encoding. SwitchTree [11] placed an RF of three trees in BMv2 with exact-match direct mapping (binary only on UNSW-NB15), Akem et al. [1] encoded RF on Tofino for encrypted-traffic classification with range/ternary entries, and Zang et al. [21] deployed DT/RF on IoT gateways through Planter using five L4-based features. Ganesan and Sarac [6] use range encoding for an ensemble on CIC-IDS2017 but only for binary classification, while attack-specific statistical approaches [9,19,12] rely on hand-tuned thresholds rather than learned boundaries. Hybrid systems [5,14,4] extract features in the data plane but run inference in the controller, reintroducing the latency that in-network processing aims to eliminate.

**Encoding and dimensionality reduction:** Exact-match encoding [11,23] requires one entry per discretized value and scales with the discretization grid; range/ternary encoding [1,6,23] collapses contiguous intervals into single rules by exploiting axis-aligned tree splits. The proposed pipeline is the first to adopt range match for both the PCA projection and the downstream classifier, yielding a compact 5119-entry raw DT and a tunable PCA ladder ( $k \in \{1, \dots, 15\}$ ) under a single encoding. Range and exact match consume different on-chip resources on programmable ASICs (range tables compile to TCAM after prefix expansion, while exact-match tables are placed in SRAM hash banks), so the entry-count comparison is on the logical rule set, not a direct TCAM saving over exact match.

As Table 1 illustrates, no existing work simultaneously combines range-match PCA encoding, range-match tree classifiers, multiclass output with four or more classes, bidirectional flow features, and a fully automated PCAP-to-P4 pipeline. Planter [23] is the closest prior work, but the differences matter in practice: it treats PCA as a standalone module and uses an exact-match-based mapping for its tree classifier in the configuration most directly comparable to the proposed design, so its table size scales with the number of distinct discretized feature values, whereas the proposed pipeline encodes *both* the PCA projection and the downstream tree classifier as range-match entries, representing each tree leaf as one rule with per-feature ranges.

### 3 Proposed Framework

The system operates in two phases. An *offline training phase* (Fig. 1) processes labeled captures into a compiled P4 program and pre-populated table entries; an *online inference phase* (Fig. 2) then executes feature extraction, dimensionality reduction, and classification entirely in the data plane at line rate. Both phases use identical fixed-point arithmetic, so the switch reproduces the offline classifier’s predictions modulo the  $b$ -bit rounding of the quantized PCA codes (Section 3.1, Step 2).

**Table 1.** Comparison of P4 based ML approaches. DP = Data Plane, Hy = Hybrid, B = Binary, M = Multiclass. DP Red. = in-data-plane dimensionality reduction.

| Work            | Inference | Match Type    | Bidir. Flow | Classes  | DP Red.                |
|-----------------|-----------|---------------|-------------|----------|------------------------|
| Xiong [20]      | DP        | Exact/Ternary | ✗           | M        | ✗                      |
| IIsy [22]       | Hy        | Exact/Ternary | ✗           | B        | ✗                      |
| Planter [23]    | DP        | Exact/Ternary | ✗           | M        | PCA <sup>a</sup>       |
| Euclid [9]      | DP        | Ternary       | ✗           | B        | ✗                      |
| CD2P [19]       | Hy        | Not applied   | ✗           | B        | ✗                      |
| HISTA [12]      | Hy        | Not applied   | ✗           | B        | ✗                      |
| CO-STOP [5]     | Hy        | Not applied   | ✗           | M        | ✗                      |
| Musumeci [14]   | Hy        | Not applied   | ✗           | B        | ✗                      |
| SwitchTree [11] | DP        | Exact         | ✓           | B        | ✗                      |
| Akem [1]        | DP        | Ternary/Range | ✗           | M        | ✗                      |
| P4DDLe [4]      | Hy        | Exact         | ✓           | B        | ✗                      |
| P4Pir [21]      | DP        | Exact         | ✗           | M        | ✗                      |
| Ganesan [6]     | DP        | Range         | ✗           | B        | ✗                      |
| <b>Ours</b>     | <b>DP</b> | <b>Range</b>  | <b>✓</b>    | <b>M</b> | <b>PCA<sup>b</sup></b> |

<sup>a</sup>Standalone module, not integrated with classifier.<sup>b</sup>Integrated with downstream classifier via range match.

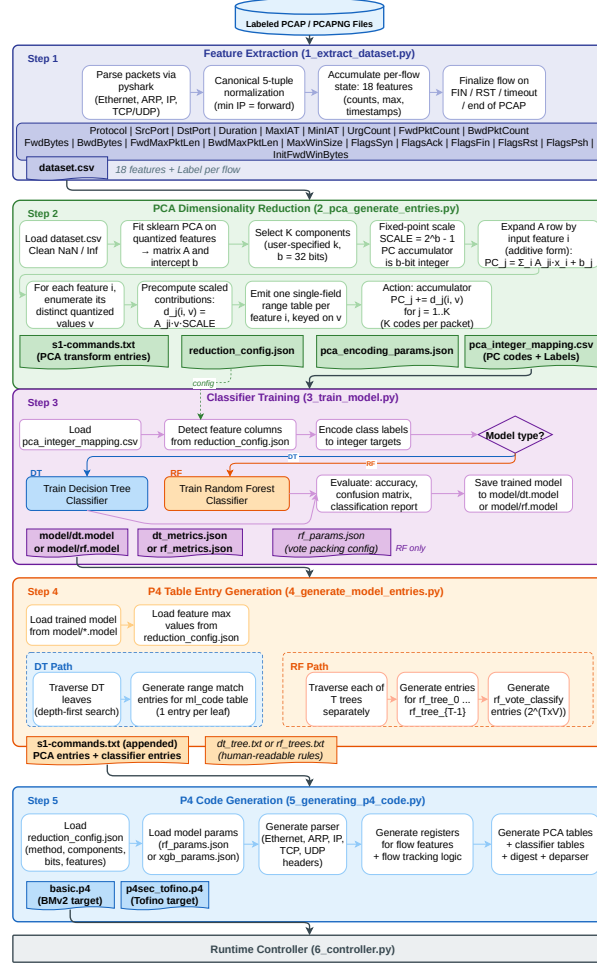
### 3.1 Offline Training Phase

The offline pipeline (Fig. 1) transforms raw PCAP captures into a compiled P4 program and a set of table-entry commands. The training data is drawn from CIC-IoT 2023 [15], a recent benchmark generated on a physical testbed covering IoT attacks across seven categories, which provides raw PCAPs (required for header-level compilation). The pipeline comprises five steps, each implemented as a standalone Python script.

#### Step 1: Feature extraction

Raw PCAP files are parsed to extract bidirectional flow features into a CSV dataset. Each flow is identified by a canonical five-tuple in which the endpoint with the numerically smaller IP address is the forward source; the same normalization is replicated in the generated P4 code. A CRC16 hash indexes the per-flow register arrays; a CRC32 hash indexes a Bloom-filter register for collision detection. A flow is finalized on a TCP FIN or RST flag, or after an idle timeout of 20 s; only finalized flows proceed to classification. Table 2 lists the 18 features produced.

The features align with the flow-level statistics used by Zeek [18] and NetFlow/IPFIX [8]; separating forward and backward counters exposes the directional asymmetries that characterize flood, scan, and amplification attacks. Flag counters and window-size extrema target TCP control-plane patterns dominant in CIC-IoT 2023; volume, timing, and port features remain protocol-agnostic.

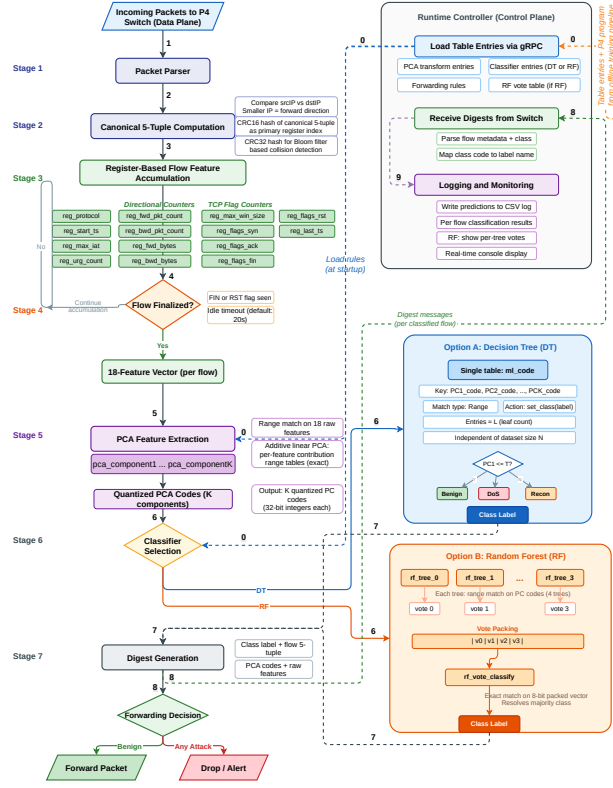


**Fig. 1.** Offline training pipeline. Five Python scripts convert raw PCAPs into a compiled P4 program with pre-populated range-match tables for the additive PCA projection and the downstream classifier.

Every feature is a count, sum, maximum, or minimum, requiring only addition and comparison, both natively supported by P4.

### Step 2: PCA dimensionality reduction

The 18 quantized features form a composite range-match key of 256 bits (Table 2), which is the operating point of the raw range-match DT baseline. PCA gives the deployer a tunable match-key dial: choosing  $k \in \{1, \dots, 18\}$  components and a  $b$ -bit quantization produces a projected key of  $k \cdot b$  bits. Each component



**Fig. 2.** In-network inference pipeline. Each finalized flow is canonicalized, accumulated into 18 bidirectional flow features, projected through the additive PCA range tables into  $K$  quantized codes, classified by a Decision Tree or 4-tree Random Forest, and emitted as a digest to the controller.

is computed by projecting the standardized feature vector and quantizing the result to a  $b$ -bit integer:

$$code_j = \text{clamp}\left(\text{round}\left(\frac{pc_j - \min_j}{range_j}(2^b - 1)\right), 0, 2^b - 1\right). \quad (1)$$

A linear PCA projection  $PC_j = \sum_{i=1}^{18} A_{j,i} x_i + \text{init}_j$  with quantized integer coefficients requires  $18 \cdot K$  multiply-accumulates per packet (126 at  $K=7$ ), which are not natively expressible inside a v1model match-action unit on the BMv2 target used in this work. The pipeline therefore exploits the additivity of the projection over features to obtain an *exact* (not approximated) range-match implementation. Step 2 fits sklearn PCA on the quantized features to obtain the rotation matrix  $A \in \mathbb{R}^{K \times 18}$  and the per-component intercept; it then expands  $A$  into one single-field range table per input feature whose action fires the  $K$  scaled

**Table 2.** Eighteen bidirectional flow features extracted in the data plane, grouped by category. The **Bits** column lists the quantized width used by the deployed P4 classifier; the per-feature widths sum to 256 bits (the raw range-match DT match-key width).

| Category  | Feature         | Description                | Bits |
|-----------|-----------------|----------------------------|------|
| Protocol  | Protocol        | IP protocol (TCP/UDP/ICMP) | 8    |
| Ports     | SrcPort         | Canonical source port      | 16   |
|           | DstPort         | Canonical destination port | 16   |
| Timing    | Duration        | First-to-last packet time  | 16   |
|           | MaxIAT          | Maximum inter-arrival time | 16   |
| Volume    | FwdPktCount     | Forward packet count       | 16   |
|           | BwdPktCount     | Backward packet count      | 16   |
|           | FwdBytes        | Forward byte count         | 16   |
|           | BwdBytes        | Backward byte count        | 16   |
|           | FwdMaxPktLen    | Max forward packet length  | 16   |
|           | BwdMaxPktLen    | Max backward packet length | 16   |
| TCP Flags | FlagsSyn        | SYN packet count           | 8    |
|           | FlagsAck        | ACK packet count           | 16   |
|           | FlagsFin        | FIN packet count           | 8    |
|           | FlagsRst        | RST packet count           | 8    |
|           | FlagsPsh        | PSH packet count           | 16   |
| Window    | MaxWinSize      | Maximum TCP window size    | 16   |
|           | InitFwdWinBytes | Initial forward TCP window | 16   |

contributions  $d_j(i, v) = A_{j,i} v \text{ SCALE\_FP}$  as accumulator increments  $\text{PC}_j += d_j$ . With  $K$  accumulators initialized to the quantized intercept and the 18 feature tables traversed once per flow, the data plane produces the same integer PCA codes that sklearn would produce offline, modulo fixed-point rounding at the chosen  $b$ -bit width. Each feature contributes one short range table whose entry count is bounded by the number of distinct quantized values for that feature, and the  $K$  codes share the same encoding as the downstream classifier. The step writes four artifacts: table-entry commands, a universal pipeline configuration file, quantization parameters, and a CSV of quantized codes with labels.

### Step 3: Classifier training

The quantized PCA codes from Step 2 train either a DT or RF classifier (Scikit-learn). The feature columns and maximum values come from the shared configuration file, so the classifier operates on exactly the representation the switch will produce.

### Step 4: Table entry generation

The trained classifier is converted to P4 range-match entries using the same DFS-with-tightest-bounds traversal as the PCA stage, but with class labels as outputs. The DT is encoded as a single range-match table whose keys are the



$K$  PCA codes; each leaf maps to one entry. A single range-match entry covers *every* feature combination within its leaf region, whereas an exact-match entry fires only when the input equals one of the stored training samples. For a tree with  $L$  leaves trained on  $N$  samples, range-match therefore produces exactly  $L$  entries with full input-space coverage within those leaves, while an exact-match deployment requires up to  $N$  entries (one per unique training feature vector) and still leaves gaps for any combination not seen at training time. On the 80/20 split adopted in this work, the raw-feature range-match DT requires 5119 entries (Section 4.2); the PCA + DT classifier table requires on the order of  $10^4$  entries for any  $k \in \{1, \dots, 15\}$ , because the PCA projection introduces additional decision boundaries that increase the leaf count relative to the raw-feature tree.

The RF classifier extends this to  $T$  trees, each encoded as a separate range-match table with the same PCA-code keys. Each tree writes its prediction into  $V = \lceil \log_2 C \rceil$  bits of a packed vote register, and a final exact-match table resolves the majority class from the concatenated  $T \times V$ -bit vector. For the four-class setting considered here ( $C=4$ ,  $V=2$ ) with  $T=4$  trees, the vote table has  $2^8=256$  entries, well within practical TCAM limits. The entire ensemble completes in a single pipeline pass.

#### Step 5: P4 code generation

A code generator reads the pipeline configuration and model parameters and emits the complete P4 program: parser, register declarations, canonical hash actions (CRC16 for indexing, CRC32 for Bloom-filter collision detection), PCA transform tables, classifier tables, digest logic, and deparser. The current implementation targets the v1model architecture and is compiled and deployed on a Raspberry Pi 4 via BMv2 (`simple_switch_grpc`); a Tofino (TNA) extension is discussed in Section 5. The outputs are passed to the runtime controller, which compiles the P4 program and installs the rules.

### 3.2 In-Network Inference Phase

Once the runtime controller loads the compiled program and table entries into the switch via gRPC, packet processing proceeds entirely in the data plane. The pipeline shown in Fig. 2 uses only addition, min/max, and conditional increments. A flow is finalized when a TCP FIN or RST flag is observed (with at least one packet) or after an idle timeout of 20 s. The finalized 18-feature vector feeds the pre-installed range-match tables: in the raw variant, a single range-match table `m1_code` keyed on the 256-bit composite feature vector returns the label directly; in the PCA variant, the projection tables first emit the  $K$  quantized PCA codes that key a second range-match classifier table, and for RF each of the  $T$  trees votes through its own range-match table into a packed vote register that an exact-match table `rf_vote_classify` resolves to the majority class. The class label, 5-tuple, PCA codes (or raw features), and per-flow metadata are then sent as a digest to the controller; the switch forwards benign traffic and drops or alerts on any attack class. The entire inference path completes in a single pipeline pass without recirculation.

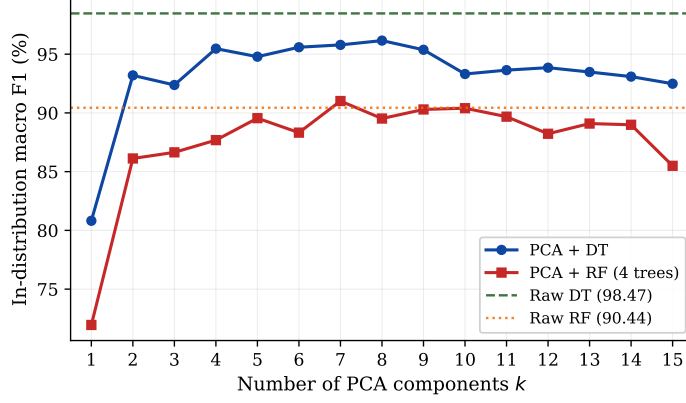
## 4 Evaluation

### 4.1 Experimental Setup

For training and in-distribution testing, the four CIC-IoT 2023 [15] categories that match the four target classes (Benign, DoS, Brute Force, Reconnaissance) are split flow-stratified 80/20, and the 20% test partition is injected through the deployed pipeline and scored against ground-truth labels, rather than evaluated offline. Cross-dataset evaluation uses a disjoint capture from the Mu-IoT [3] testbed with the same four class labels, processed through the same pipeline without retraining. Mu-IoT is chosen as the cross-dataset partner because it was independently constructed on a different physical IoT testbed with a different device population, while sharing the four-way attack taxonomy; the datasets therefore differ in distribution but not in task formulation, isolating distribution shift from task change. Within-class protocol coverage also differs: the Brute-Force class in CIC-IoT 2023 is concentrated on FTP (port 21), whereas Mu-IoT BruteForce spans HTTPS, SSH, and IRC traffic, so cross-dataset BruteForce performance directly measures generalization beyond source-specific protocols. The testbed is a Raspberry Pi 4 running BMv2 (`simple_switch_grpc`) with the v1model architecture; two network-namespace hosts on the same board inject and receive traffic via veth pairs, and labeled PCAPs are replayed with `tcpreplay`. The switch extracts the 18 bidirectional flow features, applies the additive PCA transform when enabled, and classifies in the data plane; finalized flows are sent as gRPC digests to a Python control plane that scores them against ground-truth class labels. Porting to a production ASIC such as Intel Tofino requires rewriting against the TNA architecture and the stage-packing work outlined in Section 5. Across both datasets, 32 configurations are deployed: a raw-feature baseline for the Decision Tree and a 4-tree Random Forest, plus 30 PCA settings spanning  $k \in \{1, \dots, 15\}$  at  $b=32$  for both classifiers. Macro F1 is used as the primary classification metric because both partitions are class-imbalanced (Reconnaissance contributes far fewer flows than the other classes on Mu-IoT), and macro F1 weights every class equally while penalizing both false negatives and false positives, which matches the operational concern of an IDS. Alongside macro F1, total range-match entries and classifier match-key width are reported as the P4 deployment cost. All percentages are quoted with two decimals.

### 4.2 Results

In distribution, the raw range-match DT dominates with macro F1 98.47% (overall accuracy 99.04%) using only 5 119 entries and a 256-bit composite key. None of the 30 PCA configurations matches this number, because PCA is a lossy projection and the 18 raw features are already individually discriminative on CIC-IoT. Within the PCA family the DT curve rises steeply between  $k=1$  (macro F1 80.82%) and  $k=4$  (95.47%) and plateaus across  $k \in \{4, \dots, 9\}$  at 94.79% to 96.16% (Fig. 3). PCA + RF lies approximately 5 percentage points below PCA



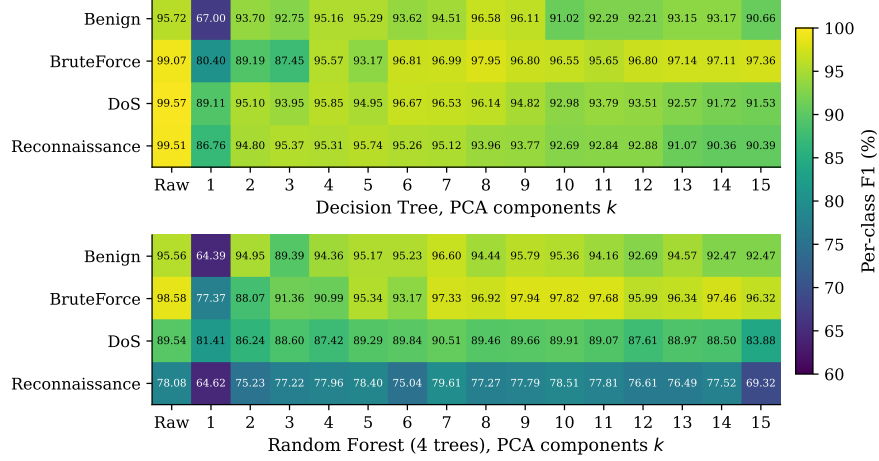
**Fig. 3.** In-distribution macro F1 versus the number of PCA components  $k$ , on the CIC-IoT 2023 test partition.

+ DT on average and stays at or below the raw RF baseline (90.44%) for every  $k$ .

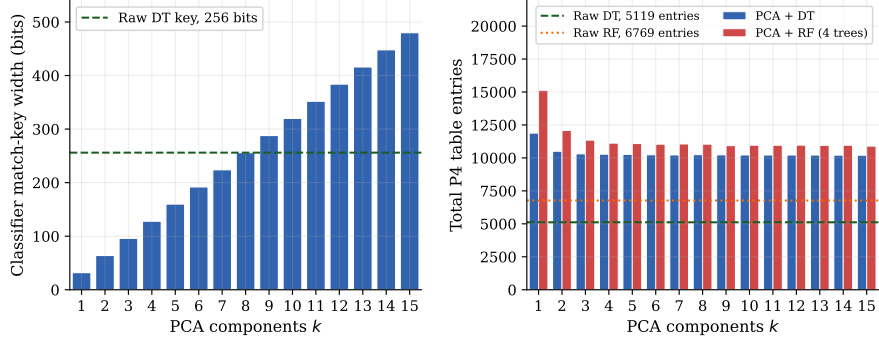
The macro F1 aggregates four distinct per-class behaviors; Fig. 4 therefore reports per-class F1 for the DT and RF families. The raw DT approaches saturation on every attack class (BruteForce 99.07%, DoS 99.57%, Reconnaissance 99.51%) with Benign at 95.72%, so the reduction from PCA is not a per-class collapse but a roughly uniform decrease as  $k$  varies. PCA + DT recovers substantially: BruteForce and DoS exceed 97% at  $k=5$  and Reconnaissance reaches 95.31% at the same  $k$ . PCA + RF performs poorly on Reconnaissance, where its best F1 across  $k \in \{1, \dots, 15\}$  is 79.61%, because the four-tree majority vote misclassifies rare scanning patterns as neighboring benign codes once the feature space is compressed.

The P4 footprint of the PCA configurations is dominated by the additive PCA transform tables. Total table entries stay between 10 200 and 11 000 for every  $k \in \{2, \dots, 15\}$ , against 5 119 for the raw DT and 6 769 for the raw RF (Fig. 5, right). The PCA + DT footprint is therefore about twice the raw DT footprint, and the PCA + RF footprint is about 1.6 times the raw RF baseline. The relative overhead of adding the projection stage is therefore moderate and decreases for the RF family, where the per-tree classifier tables already account for a larger share of the entry budget. The classifier match-key grows linearly with  $k$  at  $b=32$ , from 32 bits at  $k=1$  to 480 bits at  $k=15$ , and equals the raw DT baseline of 256 bits exactly at  $k=8$ , exceeding it for  $k \geq 9$  (Fig. 5, left). PCA therefore narrows the match-key for  $k \leq 7$  and widens it for  $k \geq 9$ , so the recommended operating point at  $k=7$  yields a narrower key than the raw DT baseline.

Under cross-dataset evaluation, the same trained models process the disjoint Mu-IoT capture through the identical P4Pi pipeline without any retraining. The raw DT degrades from 98.47% macro F1 in distribution to 15.57% on Mu-IoT,



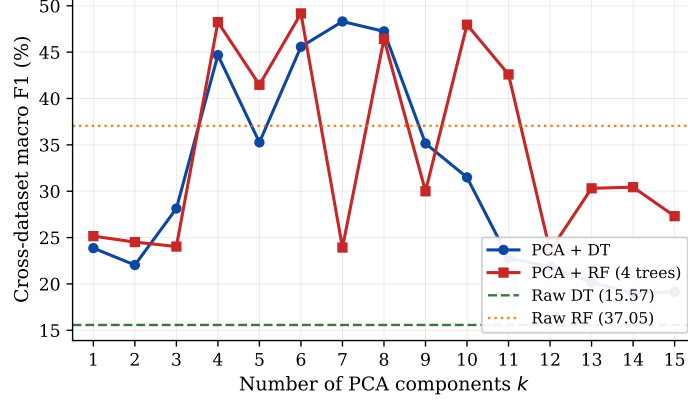
**Fig. 4.** Per-class F1 on the CIC-IoT 2023 test partition for the Decision Tree (top) and the 4-tree Random Forest (bottom). The first column ( $k$ =Raw) is the no-PCA baseline; the remaining columns are PCA settings  $k \in \{1, \dots, 15\}$ .



**Fig. 5.** Classifier match-key width (left) and total P4 table entries (right) versus the number of PCA components  $k$ .

while PCA + DT in the  $k \in \{4, \dots, 8\}$  band sustains 44.68% to 48.31% (Fig. 6), a gain of 29 to 33 percentage points. The improvement is attributable primarily to DoS detection: the raw DT fails to transfer DoS because the CIC-IoT DoS class is concentrated on a single destination port absent from the Mu-IoT testbed, while the additive PCA projection combines the 18 features into composite components before quantization, breaking the classifier's dependence on any single feature and producing multi-feature combinations whose decision boundaries prove more stable under distribution shift.

The two regimes prefer different pipelines, so the deployable operating point is selected through the standard Pareto-efficiency criterion [17,13]: a configuration

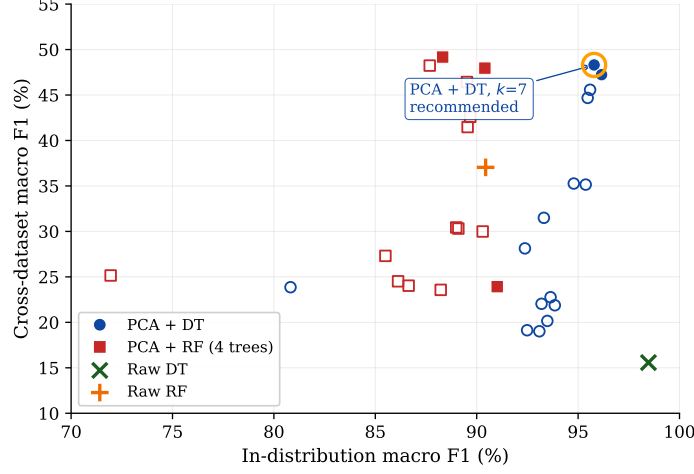


**Fig. 6.** Cross-dataset macro F1 on the Mu-IoT testbed versus the number of PCA components  $k$ .

**Table 3.** Recommended PCA + DT operating point ( $k=7$ ,  $b=32$ ) compared with the raw range-match DT baseline. All figures are measured on the Raspberry Pi 4 testbed on the CIC-IoT 2023 test partition and the Mu-IoT cross-dataset capture.

| Metric                   | Raw DT  | PCA + DT | Winner                    |
|--------------------------|---------|----------|---------------------------|
| P4 entries (total)       | 5 119   | 10 222   | Raw (about 2 times fewer) |
| Classifier key (bits)    | 256     | 224      | PCA (12.50% narrower)     |
| Rules-load time          | 10.09 s | 21.55 s  | Raw                       |
| In-distribution macro F1 | 98.47%  | 95.79%   | Raw (+2.68 pp)            |
| Cross-dataset macro F1   | 15.57%  | 48.31%   | PCA (+32.74 pp)           |

is Pareto-efficient when no other configuration improves it on every objective simultaneously, with in-distribution macro F1 and cross-dataset macro F1 as the two objectives. Fig. 7 plots all 32 configurations; four lie on the Pareto front: raw DT (98.47%, 15.57%), PCA + DT at  $k=8$  (96.16%, 47.24%), PCA + DT at  $k=7$  (95.79%, 48.31%), and PCA + RF at  $k=6$  (88.32%, 49.18%). PCA + DT at  $k=7$  is the recommended operating point: it achieves the highest cross-dataset F1 among DT-family Pareto configurations (48.31% versus 47.24% at  $k=8$ ), so  $k=8$  trades 1.07 percentage points of cross-dataset performance for only 0.37 percentage points more in-distribution an unfavourable exchange for deployment. The only Pareto configuration with higher cross-dataset F1 is PCA + RF at  $k=6$  (49.18%), but its 0.87 percentage-point cross-dataset advantage comes at the cost of 7.47 percentage points of in-distribution F1 (88.32% versus 95.79%). Additionally,  $k=7$  has a narrower classifier key than  $k=8$  (224 versus 256 bits). Table 3 summarizes this operating point against the raw DT baseline.



**Fig. 7.** Configurations plotted on the two objective axes used for operating-point selection: in-distribution macro F1 (CIC-IoT 2023) on the  $x$  axis and cross-dataset macro F1 (Mu-IoT testbed) on the  $y$  axis. Filled markers lie on the Pareto front; the highlighted point is the recommended PCA + DT at  $k=7$ .

## 5 Towards Hardware Deployment on Intel Tofino

The measurements in Section 4.2 are collected on a Raspberry Pi 4 running BMv2, which executes the v1model pipeline in software. Deploying the same design on an ASIC such as Intel Tofino is necessary for higher-tier IoT and cyber-physical workloads but imposes strict constraints on pipeline depth, on-chip memory, match resources, and action complexity [2]. The hardware pressure of the proposed pipeline comes from three sources: stateful flow storage in the feature-accumulation block (SRAM scales with the number of tracked flows times the per-flow register footprint), match-action resources in the PCA transform and classifier (range tables compile to TCAM, bounded by entry count and match-key width), and control-plane reporting via Tofino digests (triggered event-driven on flow termination, idle timeout, or attack classification rather than per packet).

The raw range-match DT keys on an 18-feature composite vector of 256 bits (Table 2); the recommended PCA + DT at  $k=7$  keys on 224 bits. Both are within the match-key width supported per stage by an RMT-style pipeline [2], of which Tofino-1 is the production instance, so match-key width is not the binding constraint and no key-width reduction is required. To make the per-stage allocation concrete, Table 4 proposes a Tofino-friendly grouping of the deployed 18 features into eight 32-bit PHV slots: each row represents one 32-bit slot holding two 16-bit fields (or four 8-bit fields in the case of the flag pairs in row 1), and the eight slots together total 256 bits, matching the BMv2 raw composite key without any additional widening introduced by the ASIC layout.

**Table 4.** Illustrative grouping of the 18 deployed features into eight 32-bit PHV slots for a Tofino port. Each row represents one 32-bit slot containing two 16-bit fields (row 1 instead packs four 8-bit flag counters). The total is 256 bits, the same as the BMv2 raw classifier key.

| Field 1                    | Bits  | Container | Field 2                    | Bits       |
|----------------------------|-------|-----------|----------------------------|------------|
| <i>Protocol + FlagsSyn</i> | 8 + 8 | 32-bit    | <i>FlagsFin + FlagsRst</i> | 8 + 8      |
| <i>SrcPort</i>             | 16    | 32-bit    | <i>DstPort</i>             | 16         |
| <i>Duration</i>            | 16    | 32-bit    | <i>MaxIAT</i>              | 16         |
| <i>FwdPktCount</i>         | 16    | 32-bit    | <i>BwdPktCount</i>         | 16         |
| <i>FwdBytes</i>            | 16    | 32-bit    | <i>BwdBytes</i>            | 16         |
| <i>FwdMaxPktLen</i>        | 16    | 32-bit    | <i>BwdMaxPktLen</i>        | 16         |
| <i>FlagsAck</i>            | 16    | 32-bit    | <i>FlagsPsh</i>            | 16         |
| <i>MaxWinSize</i>          | 16    | 32-bit    | <i>InitFwdWinBytes</i>     | 16         |
| <b>Total</b>               |       |           |                            | <b>256</b> |

The grouping is illustrative rather than measured: porting the design to Tofino has not yet been performed.

The binding constraint is therefore pipeline depth, not match-key width. An offline estimator that counts one stage per major data-dependent operation reports: 18 stages for feature extraction (one register read-write per bidirectional feature), one stage for quantization,  $k$  stages for the additive PCA transform (one per component), and one stage for the classifier, totalling 20 stages for the raw DT and 21 to 35 stages for PCA configurations ( $k=1$  to  $k=15$ ), against the 12-stage Tofino-1 ingress budget. Stage-count compaction is therefore the central hardware engineering task for the Tofino port, with the following three concrete directions identified. Three directions are proposed for the port. First, tighter feature selection retains fewer but more informative inputs, shrinking the feature-accumulation block and its stage count proportionally. Second, conditional shared-stage placement of the per-flow register arrays uses a single stateful ALU action that branches on the direction bit to update either the forward or the backward counter within one stage, rather than two separate stages. Third, multiple per-feature contribution range-match lookups are combined into compound tables with joint keys, resolving two or more features in a single stage at the cost of larger table entries. These directions are the subject of future work targeting the UfiSpace S9180-32X / BFN-T10-032D Tofino testbed.

## 6 Conclusion and Future Work

This paper has presented an automated pipeline for tree-based multiclass IoT attack detection entirely inside a P4 data plane. The classifier is encoded on 18 bidirectional flow features as a range-match table with one entry per tree leaf, and an optional additive PCA stage is itself encoded as one single-field range table per input feature. Across 32 configurations deployed on a Raspberry Pi 4,

the in-distribution and cross-dataset regimes prefer different pipelines. In distribution on CIC-IoT 2023 the raw DT achieves the highest macro F1 (98.47%, 5 119 entries, 256-bit key); on the disjoint Mu-IoT capture it degrades to 15.57% because of dataset-specific port associations that do not generalize. PCA + DT at  $k=7$  trades 2.68 percentage points of in-distribution macro F1 (95.79%, 10 222 entries) for a +32.74 percentage-point gain in cross-dataset macro F1 (48.31%), and is therefore the recommended deployable operating point. A Tofino-1 resource analysis confirms that both pipelines fit the per-stage match-key budget; stage-count compaction through the directions described in Section 5 is the identified path to full ASIC deployment. Future work will validate the pipeline on a Tofino testbed, extend the cross-dataset evaluation to UNSW-NB15, CIC-IDS 2017, and EDGE-IIoTset, and add controller-driven incremental rule updates.

**Acknowledgments.** This work was supported by the Ministry of Industry and Trade of the Czech Republic under Grant No. CZ.01.01.01/01/24\_062/0007470. It was also funded in part by the German Research Foundation (DFG, Deutsche Forschungsgemeinschaft) under Germanys Excellence Strategy-EXC 2050/2, Project ID 390696704 (Cluster of Excellence Centre for Tactile Internet with Human-in-the-Loop (CeTI), TU Dresden), and by the Federal Ministry of Education and Research of Germany in the programme of "Souverän. Digital. Vernetzt.". Joint project 6G-life, project identification number: 16KIS2413K.

## References

1. Akem, A.T.J., Fraysse, G., Fiore, M.: Encrypted traffic classification at line rate in programmable switches with machine learning (2024). <https://doi.org/10.1109/NOMS59830.2024.10575394>
2. Bosshart, P., Gibb, G., Kim, H.S., Varghese, G., McKeown, N., Izzard, M., Mujica, F., Horowitz, M.: Forwarding metamorphosis: fast programmable match-action processing in hardware for sdn **43**(4), 99110 (Aug 2013). <https://doi.org/10.1145/2534169.2486011>
3. Clinton, U.B., Hoque, N.: MU-IoT: A new IoT intrusion dataset for network and application layer attacks analysis. *IEEE Access* **12**, 166068–166092 (2024). <https://doi.org/10.1109/ACCESS.2024.3494052>
4. Doriguzzi-Corin, R., Knob, L.A.D., Mendozzi, L., Siracusa, D., Savi, M.: Introducing packet-level analysis in programmable data planes to advance network intrusion detection. *Computer Networks* **239** (2024). <https://doi.org/10.1016/j.comnet.2023.110162>
5. El-Sayed, A., Toony, A.A., Alqahtani, F., Alginahi, Y., Said, W.: Co-stop: A robust p4-powered adaptive framework for comprehensive detection and mitigation of coordinated and multi-faceted attacks in sd-iot networks. *Computers and Security* **151** (2025). <https://doi.org/10.1016/j.cose.2025.104349>
6. Ganesan, A., Sarac, K.: Adversarial evasion attack detection using ensemble classifiers in p4 data plane. *Journal of Network and Systems Management* **33**(4) (2025). <https://doi.org/10.1007/s10922-025-09948-7>
7. Gao, Y., Wang, Z.: A review of p4 programmable data planes for network security. *Mobile Information Systems* **2021** (2021). <https://doi.org/10.1155/2021/1257046>



8. Hofstede, R., eleda, P., Trammell, B., Drago, I., Sadre, R., Sperotto, A., Pras, A.: Flow monitoring explained: From packet capture to data analysis with netflow and ipfix. *IEEE Communications Surveys and Tutorials* **16**(4), 2037–2064 (2014). <https://doi.org/10.1109/COMST.2014.2321898>
9. Ilha, A.D.S., Lapolli, A.C., Marques, J.A., Gaspary, L.P.: Euclid: A fully in-network, p4-based approach for real-time ddos attack detection and mitigation. *IEEE Transactions on Network and Service Management* **18**(3), 3121–3139 (2021). <https://doi.org/10.1109/TNSM.2020.3048265>
10. Laki, S., Stoyanov, R., Kis, D., Soulé, R., Vörös, P., Zilberman, N.: P4Pi: P4 on Raspberry Pi for networking education. *SIGCOMM Comput. Commun. Rev.* **51**(3), 17–21 (2021). <https://doi.org/10.1145/3477482.3477486>
11. Lee, J.H., Singh, K.: Switchtree: in-network computing and traffic analyses with random forests. *Neural Computing and Applications* **37**(28), 23143–23154 (2025). <https://doi.org/10.1007/s00521-020-05440-2>
12. Li, D.C., Tu, H.H., Chou, L.D.: Cross-layer detection and defence mechanism against ddos and drdos attacks in software-defined networks using p4 switches. *Computers and Electrical Engineering* **118** (2024). <https://doi.org/10.1016/j.compeleceng.2024.109307>
13. Marler, R.T., Arora, J.S.: Survey of multi-objective optimization methods for engineering. *Structural and Multidisciplinary Optimization* **26**(6), 369–395 (2004). <https://doi.org/10.1007/s00158-003-0368-6>
14. Musumeci, F., Fidanci, A.C., Paolucci, F., Cugini, F., Tornatore, M.: Machine-learning-enabled ddos attacks detection in p4 programmable networks. *Journal of Network and Systems Management* **30**(1) (2022). <https://doi.org/10.1007/s10922-021-09633-5>
15. Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A.: Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors* **23**(13) (2023). <https://doi.org/10.3390/s23135941>
16. Nguyen, H.N., Nguyen, M.D., Montes De Oca, E.: A framework for in-network inference using p4 (2024). <https://doi.org/10.1145/3664476.3670453>
17. Pareto, V.: *Manuale di economia politica*. Società Editrice Libreria, Milan (1906), english translation: *Manual of Political Economy*, A. M. Kelley, New York, 1971.
18. Paxson, V.: Bro: A system for detecting network intruders in real-time (1998)
19. Wang, Y.C., Su, P.Y.: Collaborative defense against hybrid network attacks by sdn controllers and p4 switches. *IEEE Transactions on Network Science and Engineering* **11**(2), 1480–1495 (2024). <https://doi.org/10.1109/TNSE.2023.3324329>
20. Xiong, Z., Zilberman, N.: Do switches dream of machine learning?: Toward in-network classification. p. 25–33 (2019). <https://doi.org/10.1145/3365609.3365864>
21. Zang, M., Zheng, C., Dittmann, L., Zilberman, N.: Toward continuous threat defense: In-network traffic analysis for iot gateways. *IEEE Internet of Things Journal* **11**(6), 9244–9257 (2024). <https://doi.org/10.1109/JIOT.2023.3323771>
22. Zheng, C., Xiong, Z., Bui, T.T., Kaupmees, S., Bensoussane, R., Bernabeu, A., Vargaftik, S., Ben-Itzhak, Y., Zilberman, N.: Iisy: Hybrid in-network classification using programmable switches. *IEEE/ACM Transactions on Networking* **32**(3), 2555–2570 (2024). <https://doi.org/10.1109/TNET.2024.3364757>
23. Zheng, C., Zang, M., Hong, X., Perreault, L., Bensoussane, R., Vargaftik, S., Ben-Itzhak, Y., Zilberman, N.: Planter: Rapid prototyping of in-network machine learning inference. *Computer Communication Review* **54**(1), 2–20 (2024). <https://doi.org/10.1145/3687230.3687232>